LAB03: DATA ORGANIZATION

CONTENT

Objective....2
CORE Guidelines & Naming conventions....2
naming conventions....2
Core guidelines....2
Exercises....3
Ex01: Explore signed vs unsigned....3
general....3
explore various number presentations....3
explore ranges of signed and unsigned numbers....4
Ex02: Explore structs....5
GENERAL....5
positions....5
game sales....6
Ex03: Awaits Human Review....6
Ex04: ArcWork....7
GENERAL....7
structs.h....7
background....7
drawing the arcs....7
Ex05: **HANDS-ON DEMO:** AnimatedLightSabers....8
Ex06: Animated arcs....9
GENERAL....9
explore existing game functions....9
clock / timer....9
candy / lollypop....10
optional: wind mill....11
Ex07: Moving lines....11
GENERAL....11
description....11
Code Organization....11
instructions....11
Ex08: Sliding rectangles....12
GENERAL....12

Code Organization....12
description....12
Ex09: Coding Craftsmanship Challenge....13
Instructions....13
Wall of Fame....13
Submission instructions....13
Before zipping....13
Submit....13
References....14
Binary literals....14
Integer overflow....14

OBJECTIVE

At the end of these exercises, you should be able to:

- Know and work with unsigned variable types
- Getting the size of variables
- Explain the range of integer type variables and what overflow means
- Create and use structs
- Use **const** keyword in variable definitions
- Know what the difference is between local and global variables.
- Create animated graphics

We advise you to **make your own summary of topics** which are new to you.

CORE GUIDELINES & NAMING CONVENTIONS

NAMING CONVENTIONS

The **naming conventions** used in Prog1 and Prog2 promote a consistent coding style and should be followed, just as you will need to follow the conventions of any project, team, or company you end up with.

Category	Naming Convention / Pattern	Code Example
Local Variables	**l**ower**C**amel**C**asing	`float width{};` `const float innerSize{ 10.f };`
Global Variables	g_UpperCamelCasing	`int g_NumberOfEnemies{};` `const float g_WindowWidth{ 500.0f };`

Boolean Identifiers	Verb indicating boolean state (Is..., Has…)	`bool isReady{};`
		`bool g_IsAlive{};`
		`bool IsKeyPressed();`

Functions / Methods	**U**pper**C**amel**C**asing (Verb-based)	`ClearBackground(); // return type void`
Types (Struct)	**U**pper**C**amel**C**asing	`struct Circle{…};`

CORE GUIDELINES

Always check the **PROG1: Naming Conventions & C++ Core Guidelines** document for a detailed explanation about the guidelines below.
(**New guidelines** introduced in this lab are indicated in **blue**)

- ✓ **F1;** Name meaningful operations
- ✓ **F3;** Keep functions short and simple
- ✓ **ES.5;** Keep scopes small
- ✓ **ES.10;** Declare one name (only) per declaration
- ✓ **ES.20;** Always initialize an object
- ✓ **ES.23;** Prefer the {} initializer syntax
- ✓ **ES.48;** Avoid casts if possible
- ✓ **CON.1;** By default, make objects immutable
- ✓ **NL.1;** Don't say in comments what the code already says
- ✓ **NL.3;** Keep comments crisp
- ✓ **NL.5;** Don't encode type information in names
- ✓ **NL.8;** Use a consistent naming style
- ✓ **NL.9;** Use ALL_CAPS only for macros
- ✓ **NL.20;** Don't place two statements on the same line
- ✓ **NR.1;** Don't force all declarations to the top

EXERCISES

Each lab you should **create a folder** with the correct name:

- **L03_ Name_FirstName**
  - Replace **Name** and **FirstName** by your personal last and first name
    - Avoid any special characters like ` or - by replacing them with an underscore _
  - Replace **L02** with the correct lab number each week
  - All projects from this lab should be created inside this folder

-- Remember from now on, we will use the `const` *qualifier proactively when creating variables that do not change value! --* ***C++ Coding Standards- rule # 15***

```
const int maxValue{ 100 }; //good if maxValue never changes
const int diameter { radius * 2 }; // good (even if radius is not const!)
```

EX01: EXPLORE SIGNED VS UNSIGNED

GENERAL

- Create a new **CONSOLE PROJECT** called **EX01_ExploreUnSigned** in this lab's folder
- Topics needed for this exercise: signed vs unsigned, ranges, overflow

EXPLORE VARIOUS NUMBER PRESENTATIONS

Create a function that we will use to test and explore various number representations (hexadecimal, binary,..); make sure you give it a good name and don't forget to call it.

Literals are the tokens of a C++ program that represent constant values embedded in the source code.[1] E.g.: `13`, `1.4f`, `'x'`, `"NO"`, `true`, …

- Create 3 different variables of type `int`
- Each variable gets the value of 12 in a different way:
  - For the first, just use a **decimal literal** value notation.
  - For the second variable, use a **hexadecimal literal** value notation.

[1] Source: https://en.cppreference.com/cpp/language/expressions%23Literals#Literals

- For the third variable, use a **binary literal** value notation.
- *Need help with finding the hexadecimal and binary value of this number?*
- ***Try to find out how*** *you can enter hexadecimal and binary literal values in code.*
  If you cannot find it (spoiler alert), you can always check here.

```
Variable value initialized with decimal value: 12
Variable value initialized with hexadecimal value: 12
Variable value initialized with binary value: 12
```

  - ⇨ *Psssst, did you remember to use* `const` *proactively here?* ***Anything*** *that does not change value* ***after*** *initializing should be const (even if initialized with calculation)!*

EXPLORE RANGES OF SIGNED AND UNSIGNED NUMBERS

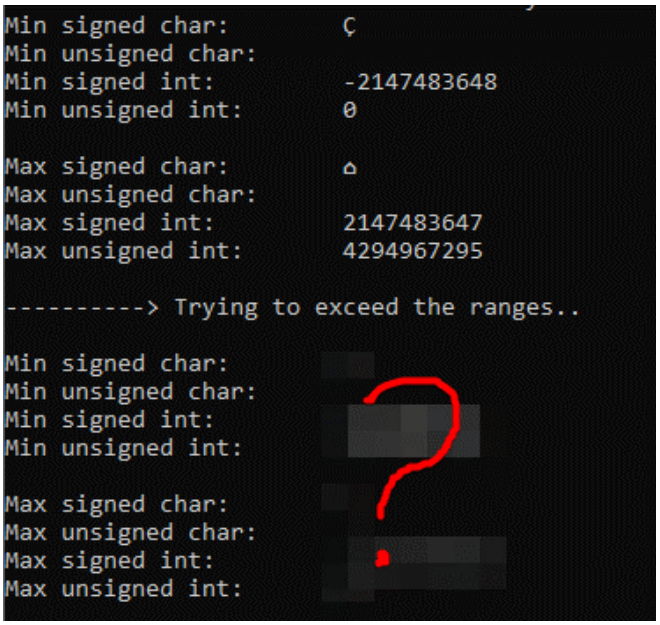
-- Table with range of value each type can store: --

Type	#bytes	Min	Max
char	1	-128	127
unsigned char	1	0	255
int	4	-2147483648 Or INT32_MIN	2147483647 or INT32_MAX
Unsigned int	4	0	4294967295 or UINT32_MAX

Create a function that will be used to check the ranges of signed and unsigned numbers. It will also test what happens when we exceed those ranges. Choose a good name and do not forget to call it.

- Define 2 variables **for each type** mentioned in the table above:
  1. A first that contains the maximum value its type can have
  2. A second that contains the minimum value its type can have
- Print all their values to the console
- Pause for a second here and try to reason: what would their value be if...
  1. We subtract exactly 1 from all variables that already contain the minimum?
  2. Add exactly 1 to all variables that already contain the maximum?

- Try it in code and print the values again to see if you were right.
  (If not, check the theory part on overflow again and/or check this reference here).



Note for Win11: *other max signed char!*

EX02: EXPLORE STRUCTS

GENERAL

- Create a new **PROG1 CONSOLE PROJECT** called **EX02_ExploreStructs** in this lab's folder
- Topics: we will create and use our own defined `struct` , also: do not forget about `const` !

POSITIONS

Create a function that will use a struct to store positions. Choose a good name, don't forget to call it.

- Given is this struct, add it to your function:

```
struct Point2f
{
	float x;
	float y;
};
```

- Create **one** variable of type `Point2f` called `position`
  - Initialize its x member with a random value between 10 and 100
  - Initialize its y member with a random value between 20 and 200
  - *Do not forget to seed before generating the random values*
  - Print their values to the console
- Next, ask which movement the user wants in the x direction. Do the same for the y direction.
  - Add this movement to the x and y position of `position` variable
  - Print the new values to the console
- Example result:

```
The initial position (x, y) is: 53, 9
-> What movement do you want in the x direction? 100
-> What movement do you want in the y direction? -100
The position after movement (x, y) is: 153, -91
```

the highlighted values indicate user input; you do not need to make any text yellow

GAME SALES

Create a function that will print the top 3 best selling videogames. Choose a good name, don't forget to call it.

- We want to use the information found here *(note: actual data might not be accurate!)*

Title	Sales	Series
Minecraft	238,000,000	*Minecraft*
Grand Theft Auto V	185,000,000	*Grand Theft Auto*
Tetris (EA)	100,000,000	*Tetris*

- Create a struct with a good name, for example `GameSale`
  - ⇨ Give it the necessary members to store the info in the first 3 columns, in other words:
  - ⇨ `title`, `sales` and `series` (choose the best type for each member)
- Create 3 variables of this type:
  1. `bestSelling` : initialize the variable with the data of the first game you see in the list
  2. `secondBest` : initialize the variable with the data of the second game you see in the list
  3. `thirdBest` : initialize the variable with the data of the third game you see in the list
- Try to use these variables to print the following to the console:
  - ➢ **Note 1:** what is highlighted in yellow should be the actual data from the above variables; use the dot `.` notation to get their values. The first line shows the title with its series between ()
  - ➢ **Note 2:** you are **not** expected to make anything yellow in your console window.

```
#1 best selling game: Minecraft (Minecraft) with 238000000 sales.
Second best: Grand Theft Auto V (185000000 sales)
Third place: Tetris (EA) (100000000 sales)
```

- *Psssst, did you remember to use* `const` *where possible in these exercises?*

EX03: AWAITS HUMAN REVIEW

Unzip the **given cmake project** with the same name as the title above.

You have been given **a piece of AI-generated C++ code**. Your task is to give it a code review and identify mistakes, bad practices, guideline violations, and logic errors.

Your goal

- Use the Core guidelines & naming conventions section at the start of this document as reference.
  - The new ones introduced in this lab are indicated in **blue**.
- For every guideline violation, **mention the corresponding guideline code** (e.g. ES.23, NL.5).
- **Write your findings as comments** above or next to the relevant line(s) of code.
- Always **explain in short** why the code is problematic, not just what is wrong.
- For each issue you find, comment out original code and **write the corrected version underneath it**.

IMPORTANT

IF YOU ARE PLANNING TO ASK AN AI TO REVIEW THE AI-GENERATED CODE FOR YOU, PLEASE SKIP THIS ASSIGNMENT ENTIRELY. YOU'LL SAVE:

- *YOUR TIME,*
- *OUR TIME,*

- *A FEW SERVERS,*
- *AND POSSIBLY AN ENTIRE FOREST WORTH OF ELECTRICITY.*

More importantly, you'll miss the actual learning goal: developing the ability of critical thinking. In a few weeks, you won't be graded on your ability to ask an AI for answers. You'll be graded on your ability to review code yourself. This will be done in a **closed-book** environment, without AI, notes, or online resources.

EX04: ARCWORK

GENERAL

- Create a new **PROG1 GAME PROJECT** called **EX04_ArcWork** in this lab's folder
- Change the window size to 800 x 400 (game.h).
- In Draw(), clear the background with `0.98, 0.87, 0.79`
- Topics: global variables, using structs from utils.h + creating own structs

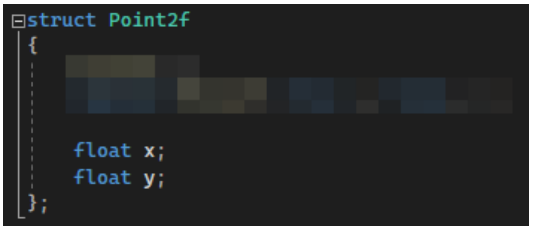
*-- We will be drawing arcs in this exercise. Did you not draw any arcs before in the game project? Then test the **DrawArc** or **FillArc** function first by drawing a simple arc using magic numbers so you are familiar with it. Remember the angles are in radians.–*

```
//example: x=200, y=300, radius=100, fromAngle=0, tillAngle=pi
FillArc(200.f, 300.f, 100.f, 100.f, 0, 3.1415f);
```

STRUCTS.H

- In the Prog1Engine project, open `structs.h`
- Check the data **members** of the following structs:

  `Point2f`
  `Color4f`
  `Ellipsef`



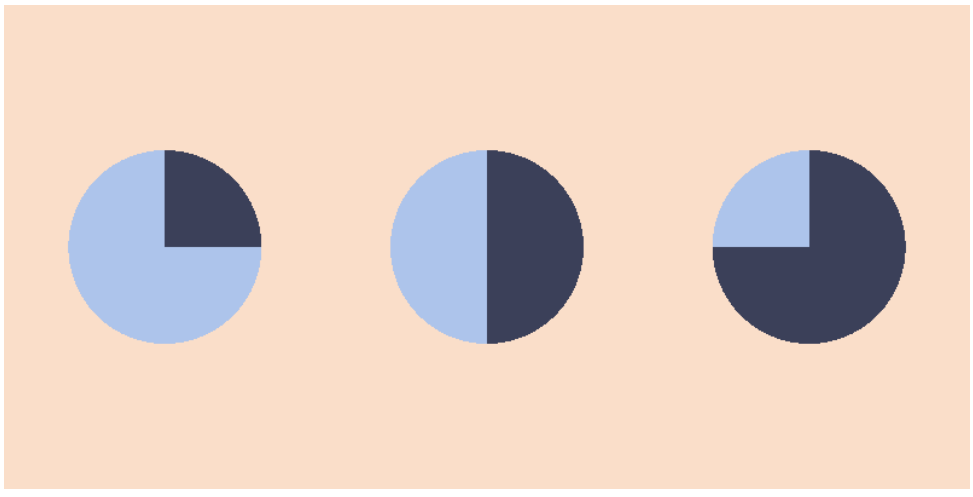
BACKGROUND

- Back to **Game.cpp**: Add this given color in the existing `Draw()` function:

```
const Color4f backgroundColor { 0.98f, 0.87f, 0.79f, 1.f };
```

- Next, use it to clear the background with this color.

DRAWING THE ARCS



Create a function to draw these 3 circles / arcs; give it a good name and do not forget to call from `Draw()`.

- You may use these given colors:

```
const Color4f powderBlue{ 0.68f, 0.77f, 0.92f, 1.f };
const Color4f delftBlue{ 0.23f, 0.25f, 0.35f, 1.f };
```

- Create a `Point2f` variable to store the **start position** of the first
  - x position: 1/6th of the window width
  - y position: halfway the height of the window

- Create 3 `Ellipsef` variables; one for each.
  1. The first one is initialized with the start position we just made, radius 80
  2. The second one is a copy of the first, but its x position is increased with 1/3rd of window width
  3. The third one is again a copy, x position further increased with 1/3rd of the window width

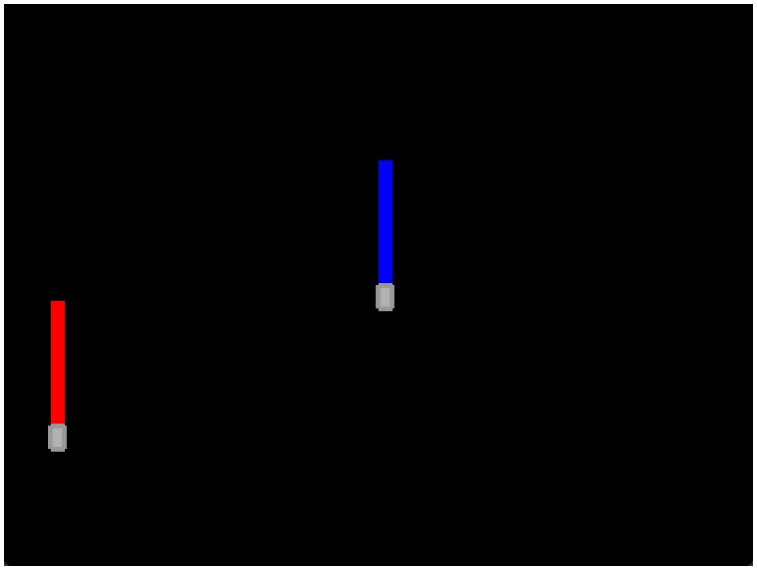
- Use these variables to draw the filled circles in `powderBlue`. Notice we have 2 options for the `FillEllipse` instruction, one where we can pass an entire `Ellipsef` variable!

```
void FillEllipse(const Ellipsef& ellipse);
void FillEllipse(const Point2f& center, float radX, float radY);
```

- Next, try to draw the arcs on it in `delftBlue`
  - Which angles do we need?

- Questions: what if we wanted...
  - A struct for an arc called `Arcf` for example, what would it look like?
  - A struct for a color combination `ColorCombo` (circle and arc color), what would it look like?
  - **[OPTIONAL!]** Try to create and use these structs in this function.

EX05: **HANDS-ON DEMO:** ANIMATEDLIGHTSABERS



In this demo, we cover:

- The need for **global variables** *only* to store information that must **persist between frames**.
- Creating a **simple frame-by-frame animation**.
- Organizing code into meaningful **helper functions** and using the correct **game functions**.

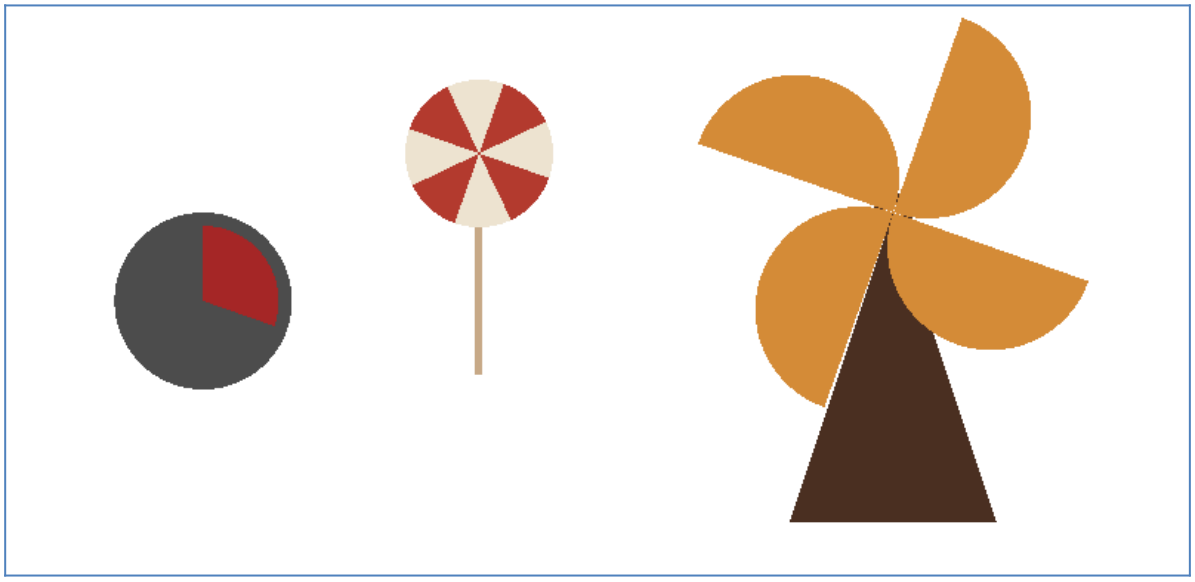
***If you reach this part before the demo**, feel free to try it independently already.*

EX06: ANIMATED ARCS

GENERAL

- Create a new **PROG1 GAME PROJECT** called **EX05_AnimatedArcs** in this lab's folder
- If you want to use the exact same colors as the screencasts, check colors.txt for color values.

*-- We start using **global variables** to create animations. Check the theory part and Demo files if necessary before you continue. Do not forget about the g_ naming convention! --*



*See also: **animatedArcs.png** on Leho to see the animation in action.*

EXPLORE EXISTING GAME FUNCTIONS

Until now, we have only worked in the Draw() game function. To create animations properly, we need to use additional game functions.

- Go the **README.md** in your game project and click on Open preview if necessary.
- Find the section "**Game Functions – Overview**" and get yourself familiar with the following functions:
  - `Start()`, `Draw()` and `Update(float elpasedSec)`
- **Think First:** Before writing code during this exercise, take a moment to decide:
  - Which tasks belong in `Start()`?
  - Which tasks belong in `Draw()`?
  - Which tasks belong in `Update()`?

CLOCK / TIMER



We will create a timer animation similar to the one in the given screencast. Note that the arc just keeps on growing larger than 1 full circle. This is visible because the red color value is slightly transparent (alpha value of RGBA < 1.f).

Extra: By pressing a key or a mouse button, the animation restarts.

See instructions below.

Analyse the Animation ⇨ Variables

Before coding, inspect the timer and think about:

- What information must be **remembered between frames**?
- What values determine the **current appearance** of the timer?
- Which values **change** over time?

⇨ Use **meaningful names** for all variables! You will need to distinguish them later from other animations.
⇨ Only information that must **persist between frames** should be **global** (e.g. current progress of animation)!
⇨ Values that can be recalculated or are only needed while drawing should remain **local** (e.g. colors).
⇨ Remember to make *anything* that does not change **const**!

Organize Your Code ⇨ Functions

Identify the different tasks involved in creating this animation and group the code into meaningful functions.
A good solution will likely contain helper functions similar to:

- InitializeTimer()
- DrawTimer()
- UpdateTimer()

⇒ You may choose different names, as long as they clearly describe their purpose.

⇨ **Declare** your helper functions in `Game.h` and **define** them in `Game.cpp`.
⇨ **Call** your functions from the correct **game functions**.
⇨ Implement **Step by Step;** do not try to build the entire animation at once:

1. Create the variables and initialise them.
2. Display the timer in its starting state.
3. Add the animation behaviour.
4. Verify that the animation updates correctly.
5. …

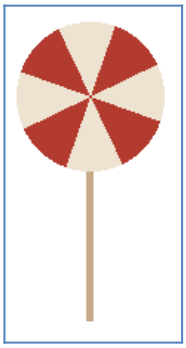
Animation Behaviour

The red progress section should gradually increase over time.
Hint: Animate the arc's end angle. Make sure it starts on top, like an actual timer. Use a very small increase per frame, such as 0.01f.

Extra: Try to reset the animation when either a key or a mouse button is pressed.

⇨ Create a function for this behavior so you can call it from anywhere you want.
⇨ **Hint:** use the **event** game functions such as `OnKeyDownEvent`.

CANDY / LOLLYPOP



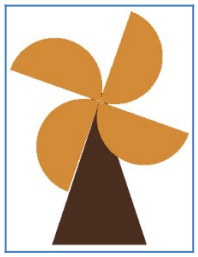
Now try to follow the same steps as with the clock / timer to create the lollypop with rotating candy.

Hint: there are only 4 arcs, drawn on a full circle in another color.

Make sure you choose names for your variables and functions that clearly distinguish themselves from the clock / timer.

Do **not** put drawing, updating, .. code directly in the game functions like `Draw()`; instead, create your own functions and call them from there.

OPTIONAL: WIND MILL



NOTE: *This exercise is* ***optional*** *and aimed at students who are not afraid of an extra challenge and feel familiar with the content already.*
This exercise ***not*** *required knowledge and it will* ***not*** *be part of an assessment or exam.*

Try to create a windmill-like animation (see screencast) by making the arcs rotate around a central point. You may use trigonometry (`sinf` and `cosf`) to calculate the position of the arc centres.

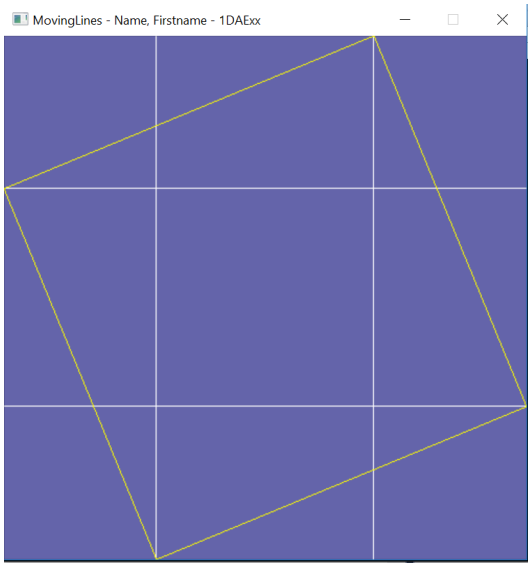
EX07: MOVING LINES

GENERAL

- Create a new **PROG1 GAME PROJECT** called **EX06_MovingLines** in this lab's folder
- Set the window size to 500 x 500 and don't forget to set build dependencies and title
- Topics: global variables, animations

DESCRIPTION

The end result should look like this:



See also: ***movingLinesAnimation.png*** *on Leho to see the animation in action.*

CODE ORGANIZATION

- Keep game functions short.
- Use helper functions for drawing, updating and initialization.
- `Draw()` should only contain drawing-related calls. `Update()` should only contain update-related calls.
  - See **README.md** in your game project.
- Choose meaningful function names (ones your classmates would understand).

INSTRUCTIONS

- Before writing code, first determine:
  - Which information must (**not**) be remembered between frames? *(Guideline* ***ES.5****!)*
  - Which values can be calculated from that information? *(= local, in draw for example)*
- ***USE A SINGLE ANIMATION VARIABLE TO CONTROL THE POSITION OF ALL MOVING LINES.***

- Make sure this variable increases with 1 each frame, but never bigger than the window size
  - You can assume the window will always be a square
  - If the value becomes to big, it resets back to 0.
  - ⇨ You may use either **modulo %** or an **if** statement Draw the white lines:
  1. One vertical line moves **from left to right**
  2. A second vertical line moves opposite; **from right to left**
  3. A horizontal line moves **from top to bottom**
  4. Another horizontal line moves **from bottom to top**

  Note: the lines do **not** switch directions, it only seems that way because they reset.

- Determine which endpoints need to be connected to reproduce the yellow shape shown in the screenshot and screencast.

EX08: SLIDING RECTANGLES

GENERAL

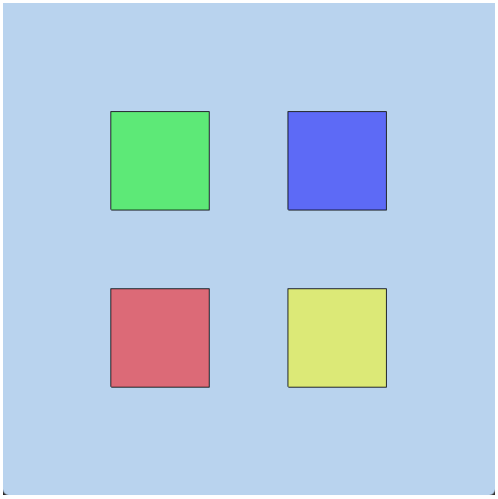
- Create a new **PROG1 GAME PROJECT** called **EX07_SlidingRectangles** in this lab's folder
- Make sure the window is a square

CODE ORGANIZATION

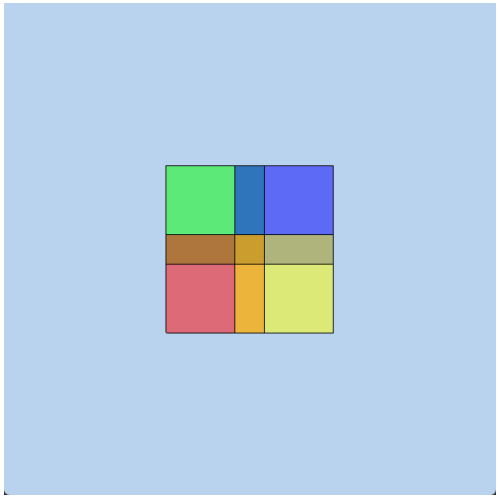
- Keep game functions short.
- Use helper functions for drawing, updating and initialization.
- `Draw()` should only contain drawing-related calls. `Update()` should only contain update-related calls.
  - See **README.md** in your game project.
- Choose meaningful function names (ones your classmates would understand).

DESCRIPTION

- Do not forget to create/call functions for every part of this assignment (drawing, updating,..)
- In this project you'll draw **4 semi-transparent squares** that slide between opposite window corners at a speed of 1 pixel per frame in both horizontal and vertical direction.
- They reset when they reach the sides.
- At the end of this exercise, you should have a screen that looks like this:



Moving rectangles, not overlapping.



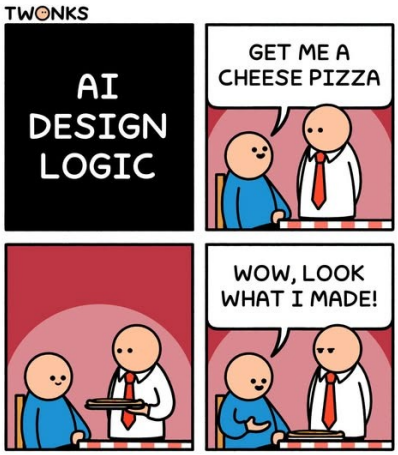
Moving rectangles while overlapping.

EX09: CODING CRAFTSMANSHIP CHALLENGE

DO NOT SKIP THIS EXERCISE; EXPERIMENTING WITH CODE IN A SELF-MADE PROJECT IS BY FAR THE BEST WAY TO MASTER PROGRAMMING!

INSTRUCTIONS

Create a **Prog1 Game Project** called **CodingCraft03** that demonstrates **animations**.



Create a **small original application** that demonstrates the concepts we've covered so far.

- ✓ Create **your own** original project.
- ✓ **Only** use **concepts that have been covered** in labs already.
- ✓ Write code you can explain yourself.
- x **No** AI-generated code.
- x **No** copied solutions from the internet.
- x **No** copying another student's work.

For example, this week: draw your favorite logo or character using basic shapes or find inspiration by searching for daily minimal in Google images.

WALL OF FAME

Every week, some projects will earn a place on the lab's Wall of Fame.

A project does not need to be complex to deserve a spot. Original ideas, clever concepts, fun themes, polished outputs, and well-written code are just as valuable. Projects are shown **anonymously** so you can choose whether you reveal yourself as the creator or not.

The **goal** is to **learn, experiment, and become a better programmer**. The Wall of Fame only works if every project genuinely reflects your own work. **Let your imagination run wild and work on a programming project YOU love!**

SUBMISSION INSTRUCTIONS

You must submit **two files**:

- A **zip file** containing your **complete lab folder** of this week with all exercises.
- **CodingCraft03.zip** containing only your Coding Craftsmanship Challenge project.

BEFORE ZIPPING

For every project:

- **Close** Visual Studio.
- Delete the **out** folder.
- Delete the hidden **.vs** folder.

SUBMIT

- Upload **both files** with the correct assignment before the start of next week's first lab.
- Make sure to complete the **quiz**.

- *These steps are required to unlock the next lab assignment!*

⚠️ Your submission may be discussed during a code review. Make sure you understand and can explain your own code.

REFERENCES

BINARY LITERALS

https://en.cppreference.com/w/cpp/language/integer_literal

INTEGER OVERFLOW

http://www.cplusplus.com/articles/DE18T05o/

https://www.ima.umn.edu/~arnold/disasters/ariane.html